Power BI – Complete Study Notes

---

1. Introduction to Power BI

Power BI is a business analytics / Business Intelligence (BI) tool developed by Microsoft that allows users to visualize data, connect to multiple data sources, transform raw data, build data models, and share interactive reports and dashboards.

1.1 Why Power BI?

- Connects to 100+ data sources (Excel, SQL Server, Web, SharePoint, cloud services, APIs, etc.)
- Free/low-cost compared to other BI tools (Tableau, Qlik)
- Easy drag-and-drop interface — no heavy coding required for basics
- Powerful data transformation engine (Power Query)
- Powerful analytical language (DAX) for calculations
- Interactive, shareable dashboards and reports
- Native integration with Microsoft ecosystem (Excel, Azure, Teams, SharePoint)

1.2 Components of Power BI

Component	Purpose
Power BI Desktop	Windows application used to connect, transform, model, and build reports (free)
Power BI Service (app.powerbi.com)	Cloud-based platform to publish, share, and collaborate on reports/dashboards
Power BI Mobile	App to view reports/dashboards on phone/tablet
Power BI Report Server	On-premise server for organizations that can't use the cloud
Power BI Gateway	Bridges on-premise data sources with Power BI Service for scheduled refresh
Power BI Embedded	Allows developers to embed Power BI visuals into custom applications
Power BI Report Builder	For creating paginated (pixel-perfect, printable) reports

1.3 Power BI Desktop – Workflow Areas

Power BI Desktop has **3 main views** (icons on the left sidebar):

1. **Report View** – Build visuals, charts, and pages
2. **Data View** – See the raw data in tabular form after loading
3. **Model View** – See and manage relationships between tables

1.4 The Power BI Workflow (End-to-End)

Connect to Data → Transform Data (Power Query) → Model Data (Relationships/DAX)

→ Visualize Data (Reports) → Publish (Power BI Service) → Share/Collaborate

---

2. Importing Data into Power BI

2.1 How to Import Data

1. Open Power BI Desktop → **Home tab → Get Data**
2. Choose data source category: File, Database, Azure, Online Services, Other
3. Select the specific source (Excel, CSV, SQL Server, Web, SharePoint list, etc.)
4. Enter credentials/connection details if prompted
5. Preview the data in the **Navigator** window
6. Click **Transform Data** (opens Power Query Editor) or **Load** (loads directly)

2.2 Common Data Sources

- **Files**: Excel, CSV, XML, JSON, PDF, Text, Folder (combine multiple files)
- **Databases**: SQL Server, MySQL, Oracle, PostgreSQL, Access
- **Cloud/Online**: SharePoint Online List, Azure SQL, Dataverse, Salesforce, Google Analytics
- **Web**: Any public web page/table via URL
- **Other**: OData Feed, ODBC/OLE DB, Blank Query, R/Python scripts

2.3 Data Connectivity Modes

Mode	Description
Import	Data is copied into Power BI's in-memory engine (VertiPaq). Fast performance but needs manual/scheduled refresh. Most common mode.
DirectQuery	Power BI sends live queries to the source; no data is stored locally. Always up-to-date but slower, and DAX has some limitations.
Live Connection	Connects directly to Analysis Services/Power BI datasets; visuals only, no modeling allowed locally.
Composite Model	Mix of Import and DirectQuery tables in the same model.

2.4 Refreshing Data

- **Manual refresh**: Click "Refresh" in Desktop or Service
- **Scheduled refresh**: Set up in Power BI Service (up to 8 times/day on Pro)
- **On-premises Gateway**: Required if the data source is local/on-premise and needs scheduled cloud refresh

---

3. Power Query (Data Transformation Layer)

Power Query is the **ETL (Extract, Transform, Load)** engine of Power BI. It uses the **"M" language** (Power Query Formula Language) behind the scenes, though most transformations can be done with the UI (no-code).

3.1 Opening Power Query Editor

Home tab → **Transform Data** → opens a separate window with:

- **Queries pane** (left) – list of all tables/queries
- **Data preview** (center) – view of current data
- **Query Settings pane** (right) – shows **Applied Steps** (every transformation is recorded as a step, like a macro)

3.2 Key Concept: Applied Steps

Every transformation you perform (removing a column, filtering rows, changing a data type) is recorded as a **step** in the Applied Steps list. This makes Power Query:

- Repeatable (re-runs automatically every refresh)
- Auditable (you can click on any step to see data at that point)
- Editable (steps can be reordered, edited, or deleted)

3.3 Common Power Query Transformations

Transformation	Use
Remove Columns / Choose Columns	Keep only relevant fields
Remove Duplicates	Clean duplicate rows
Remove Rows / Filter Rows	Remove blanks, errors, or apply conditions
Change Data Type	Text, Whole Number, Decimal, Date, Date/Time, Boolean
Split Column	By delimiter, number of characters, positions
Merge Columns	Combine multiple columns into one
Group By	Aggregate data (Sum, Count, Average) similar to SQL GROUP BY
Pivot / Unpivot Columns	Reshape data between wide and long formats (Unpivot is very commonly used to normalize data)
Replace Values	Find and replace specific values
Fill Down / Fill Up	Fill blank cells with value from above/below
Add Custom Column	Create new column using M formulas
Add Conditional Column	If-then-else logic via UI
Merge Queries	Join two tables (like SQL JOIN – Left, Right, Inner, Full, Anti joins)
Append Queries	Stack rows from multiple tables (like SQL UNION)

Transformation	Use
Extract	Pull text before/after delimiter, length, etc.
Trim / Clean / Capitalize	Text cleanup

3.4 Power Query M Language (Advanced)

- Every query is actually M code; you can view/edit it via **Advanced Editor**
- Basic structure:

```
let
   Source = Excel.Workbook(File.Contents("C:\Data\Sales.xlsx"), null, true),
   Sales_Sheet = Source{[Item="Sales",Kind="Sheet"]}[Data],
   ChangedType = Table.TransformColumnTypes(Sales_Sheet,{{"Date", type date}})
in
   ChangedType
```

- Useful when built-in UI options aren't enough (custom logic, functions, parameters)

3.5 Parameters & Functions in Power Query

- **Parameters**: Reusable values (e.g., file path, date range) that can be changed without editing every query
- **Custom Functions**: Reusable M code blocks, useful when combining/transforming multiple similar files (e.g., "Combine Files" from a folder)

---

4. Data Modeling

Before writing DAX, understanding the **model** is essential.

4.1 Relationships

- Created automatically (if detected) or manually in **Model View**
- Types: **One-to-Many** (most common), One-to-One, Many-to-Many
- Has a **Cardinality**, **Cross-filter direction** (Single or Both), and **Active/Inactive** status

4.2 Star Schema (Best Practice)

- **Fact table**: Contains transactional/numeric data (Sales, Orders) — many rows
- **Dimension tables**: Contains descriptive data (Product, Customer, Date, Region) — connected to fact table
- Star schema improves performance and makes DAX simpler compared to a flat single table

4.3 Date Table

- A dedicated **Date/Calendar table** is considered a best practice for any time-based analysis (needed for Time Intelligence DAX functions)
- Can be created manually via Power Query or DAX (CALENDAR, CALENDARAUTO)

---

5. DAX (Data Analysis Expressions)

DAX is a **formula language** used in Power BI (also Excel Power Pivot, SSAS Tabular) to create **calculated columns, measures, and calculated tables**.

5.1 DAX Objects: Column vs Measure vs Table

Type	Description	Computed
Calculated Column	New column added to a table, computed row-by-row	At data refresh, stored in the model (uses memory)
Measure	Aggregated calculation, computed dynamically based on context (filters/slicers applied)	At query/render time (doesn't use extra storage)
Calculated Table	Entirely new table generated via DAX	At data refresh

Rule of thumb: Use **Measures** for aggregations (SUM, AVERAGE, ratios, KPIs) — they are dynamic and efficient. Use **Calculated Columns** only when you need row-level values (e.g., categorizing each row).

5.2 DAX Syntax Basics

Total Sales = SUM(Sales[SalesAmount])

Profit Margin % = DIVIDE(SUM(Sales[Profit]), SUM(Sales[SalesAmount]), 0)

- Table references: TableName[ColumnName]
- Measures don't need a table prefix when referenced: [Total Sales]

5.3 Context in DAX (Most Important Concept)

- **Row Context**: Exists when a formula evaluates row-by-row (e.g., calculated columns, iterator functions like SUMX)
- **Filter Context**: The set of filters applied due to slicers, visual filters, or rows/columns in a chart/table — this is what makes measures "dynamic"
- **CALCULATE()** is the most powerful DAX function — it modifies the filter context

5.4 Important DAX Function Categories & Examples

a) Aggregation Functions

SUM(), AVERAGE(), MIN(), MAX(), COUNT(), COUNTA(), COUNTROWS(), DISTINCTCOUNT()

b) Iterator (X) Functions – evaluate row-by-row then aggregate

Total Revenue = SUMX(Sales, Sales[Qty] * Sales[UnitPrice])

AVERAGEX(), MAXX(), MINX(), COUNTX(), RANKX()

c) Filter Functions

CALCULATE(SUM(Sales[SalesAmount]), Sales[Region] = "North")

FILTER(Sales, Sales[Qty] > 10)

ALL(), ALLEXCEPT(), ALLSELECTED() -- remove/keep filters

d) Logical Functions

IF(), SWITCH(), AND(), OR(), NOT()

```
Category = SWITCH(TRUE(),
   Sales[Amount] > 10000, "High",
   Sales[Amount] > 5000, "Medium",
   "Low")
```

e) Text Functions

CONCATENATE(), LEFT(), RIGHT(), MID(), LEN(), UPPER(), LOWER(), TRIM(), FORMAT()

f) Date & Time Intelligence Functions (need a proper Date table)

TOTALYTD(), TOTALQTD(), TOTALMTD()

SAMEPERIODLASTYEAR(), DATEADD(), PREVIOUSMONTH(), PREVIOUSYEAR()

DATEDIFF(), DATESBETWEEN()

```
YTD Sales = TOTALYTD(SUM(Sales[SalesAmount]), 'Date'[Date])
Sales LY = CALCULATE(SUM(Sales[SalesAmount]), SAMEPERIODLASTYEAR('Date'[Date]))
```

g) Relationship Functions

RELATED() -- pulls a column from a related table (used in calculated columns)

RELATEDTABLE()

USERELATIONSHIP() -- activates an inactive relationship temporarily

h) Table Functions

CALENDAR(), CALENDARAUTO(), SUMMARIZE(), ADDCOLUMNS(), VALUES(), DISTINCT()

i) Information Functions

ISBLANK(), ISERROR(), HASONEVALUE(), ISFILTERED()

5.5 Common Real-World DAX Measures

Total Sales = SUM(Sales[SalesAmount])

Total Orders = DISTINCTCOUNT(Sales[OrderID])

Profit % = DIVIDE([Total Profit], [Total Sales], 0)

Rank by Sales = RANKX(ALL(Product[ProductName]), [Total Sales])

Running Total = CALCULATE([Total Sales], FILTER(ALLSELECTED('Date'), 'Date'[Date] <= MAX('Date'[Date])))

---

6. Visualizing Data / Charts in Power BI

Once data is modeled, drag fields onto the **Report canvas** and choose a visual from the **Visualizations pane**.

6.1 Common Chart Types & When to Use Them

Visual	Best Used For
Bar / Column Chart	Comparing categories (e.g., sales by region)
Line Chart	Trends over time (e.g., monthly revenue)
Area Chart	Trend + magnitude, cumulative comparisons
Pie / Donut Chart	Proportion of a whole (use sparingly — max 5-6 categories)
Card / Multi-row Card	Single KPI number (e.g., Total Sales)
Table / Matrix	Detailed, tabular data; Matrix supports row/column grouping and drill-down like a pivot table
Treemap	Hierarchical part-to-whole comparison
Scatter / Bubble Chart	Relationship/correlation between two-three numeric variables
Waterfall Chart	Sequential increase/decrease (e.g., profit bridge)
Funnel Chart	Stages of a process (e.g., sales pipeline conversion)
Gauge	Progress toward a goal/target
KPI Visual	Value vs target trend indicator
Map / Filled Map	Geographic data (by country, state, city, coordinates)
Slicer	Interactive filter control for the user
Decomposition Tree (AI visual)	Break down a metric across multiple dimensions interactively
Key Influencers (AI visual)	Auto-detect factors influencing a metric
Q&A Visual	Natural language query box

6.2 Building & Formatting Visuals

1. Select a visual type from the Visualizations pane
2. Drag fields into **Values, Axis, Legend, Tooltips** wells
3. Use the **Format pane** (paint roller icon) to customize: colors, data labels, titles, axis, gridlines
4. Use **Analytics pane** to add trend lines, average lines, forecasting

6.3 Interactivity Features

- **Slicers**: Dropdown/list/date-range filters on the report page
- **Filters pane**: Page-level, report-level, or visual-level filters
- **Cross-filtering/highlighting**: Clicking one visual filters/highlights others automatically
- **Drill Down/Up**: Explore hierarchies (Year → Quarter → Month → Day)
- **Drill Through**: Right-click a data point to jump to a detailed page filtered to that context
- **Bookmarks**: Save a specific view/filter state of the report, useful for storytelling and toggle buttons
- **Tooltips**: Custom report pages that appear on hover
- **Buttons & Navigation**: Create custom navigation between report pages

6.4 Dashboards vs Reports (Power BI Service)

Report	Dashboard
Multi-page, built in Desktop, highly interactive	Single page, built in Service, pinned from reports
Has filters, slicers, drill-through	Mostly static tiles ("pinned" visuals) with live data
Can connect to one dataset	Can pin visuals from multiple reports/datasets

---

7. Publishing & Sharing (Power BI Service)

1. **Publish**: File → Publish → choose a **Workspace** in Power BI Service
2. **Workspaces**: Collaborative spaces where teams build/share content (roles: Admin, Member, Contributor, Viewer)
3. **Sharing options**: Share report/dashboard link, publish to web (public), export to PowerPoint/PDF, embed in Teams/SharePoint
4. **Apps**: Package multiple reports/dashboards into a polished "app" for end users
5. **Row-Level Security (RLS)**: Restrict data visibility per user/role using DAX filters (e.g., a regional manager sees only their region's data)
6. **Gateway**: Required for scheduled refresh of on-premise data sources

---

8. Topics Often Missed (Good to Know)

- **Power BI Pro vs Premium vs Free**: Licensing differences (sharing limits, dataset size limits, refresh frequency)
- **Aggregations & Composite Models**: For handling very large datasets efficiently
- **Performance Analyzer**: Built-in tool in Desktop to check which visuals/DAX are slow
- **DAX Studio & Tabular Editor**: External free tools for advanced DAX debugging and model editing
- **Power BI Copilot/AI features**: Natural language Q&A, auto-generated summaries (availability depends on license)
- **Custom Visuals**: Import extra chart types from the Microsoft AppSource marketplace
- **Themes**: Apply consistent color/branding themes (JSON-based custom themes supported)
- **Paginated Reports (Report Builder)**: For pixel-perfect, printable reports (e.g., invoices)
- **Dataflows**: Reusable Power Query logic stored in the cloud, shareable across multiple reports
- **Incremental Refresh**: Refreshing only new/changed data instead of the whole dataset (Premium feature, huge performance gain)

- **Deployment Pipelines**: Dev → Test → Production workflow for enterprise report management
- **Version Control**: .pbix files can be tracked in Git; Power BI Projects (.pbip) format allows better source control
- **Error Handling in Power Query**: try...otherwise in M for handling transformation errors gracefully
- **Naming Conventions & Documentation**: Best practice to name measures/tables clearly and hide unnecessary columns from the report view for a cleaner model

---

9. Quick Summary Cheat-Sheet

Stage	Tool/Feature	Purpose
Connect	Get Data	Import from various sources
Clean/Transform	Power Query Editor	ETL, shaping data
Model	Model View, Relationships	Build star schema
Calculate	DAX (Measures/Columns)	Business logic & KPIs
Visualize	Report View, Visuals	Charts, dashboards
Share	Power BI Service	Publish, collaborate, secure
